

Aprendizado Profundo

Diffusion Models

Lucas Silveira Kupssinskü

Machine Learning Theory and Applications Laboratory
PUCRS

junho de 2026

Visão Geral

1. Modelos Geradores – Revisão
2. Visão Geral do Processo de Difusão
3. Encoder: Processo de Difusão
4. Kernel de Difusão
5. Distribuição Condicional da Difusão
6. Decoder
7. Treinamento e ELBO
8. Reparametrizando a Loss e a Rede
9. Treinamento e Amostragem
10. Implementação e Extensões
11. Panorama

Overview

1. Modelos Geradores – Revisão
2. Visão Geral do Processo de Difusão
3. Encoder: Processo de Difusão
4. Kernel de Difusão
5. Distribuição Condicional da Difusão
6. Decoder
7. Treinamento e ELBO
8. Reparametrizando a Loss e a Rede
9. Treinamento e Amostragem
10. Implementação e Extensões
11. Panorama

Modelos Geradores

- O objetivo de um **modelo gerador** é aprender uma **distribuição de probabilidade** sobre uma variável X usando dados observados $\{x^{(i)}\}_{i=1}^N$.
- A densidade $p_X(x)$ é parametrizada por θ .
- Já vimos:
 - **GANs** (jogo min-max),
 - **Normalizing Flows** (transformações invertíveis),
 - **VAEs** (encoder/decoder + ELBO).

A próxima ideia: *Diffusion Models*¹

- Definir um *encoder* **sem parâmetros treináveis** que degrada uma imagem adicionando ruído gaussiano de maneira sequencial.
- Treinar um *decoder* para **retirar o ruído** adicionado.

Intuição

Aprender a remover ruído um pequeno passo de cada vez é mais fácil que aprender a gerar uma imagem inteira do zero. Empilhando muitos passos pequenos, conseguimos transformar ruído puro em uma amostra realista.

¹Jonathan Ho, Ajay Jain e Pieter Abbeel. “Denoising Diffusion Probabilistic Models”. Em: [NeurIPS](#). vol. 33. 2020, pp. 6840–6851.

Overview

1. Modelos Geradores – Revisão
- 2. Visão Geral do Processo de Difusão**
3. Encoder: Processo de Difusão
4. Kernel de Difusão
5. Distribuição Condicional da Difusão
6. Decoder
7. Treinamento e ELBO
8. Reparametrizando a Loss e a Rede
9. Treinamento e Amostragem
10. Implementação e Extensões
11. Panorama

Visão Geral – Difusão

- O objetivo de um modelo gerador é aprender uma **distribuição de probabilidade** sobre uma variável aleatória $\mathbf{X}$ usando um conjunto de dados observados $\{x^{(i)}\}_{i=1}^N$ com uma densidade de probabilidade $p_X(x)$ parametrizada por θ
- A próxima ideia é:
 - Definir um *encoder* **sem parâmetros** treináveis que degrada uma imagem adicionando ruído gaussiano de maneira sequencial
 - O *decoder* será treinado para retirar o ruído adicionado
- No processo de difusão (*forward*) adicionamos ruído a cada passo até a imagem ficar indistinguível do ruído.
- No processo de *denoising* (decoder) treinamos uma rede neural para **remover um passo de ruído**.

Visão Geral – Roteiro

Vamos ver em detalhes:

- A **difusão** (encoder).
- O **denoising** (decoder).
- O **treinamento** (ELBO).
- A **reparametrização** para estimar ruído.
- Detalhes de **implementação**.

Overview

1. Modelos Geradores – Revisão
2. Visão Geral do Processo de Difusão
- 3. Encoder: Processo de Difusão**
4. Kernel de Difusão
5. Distribuição Condicional da Difusão
6. Decoder
7. Treinamento e ELBO
8. Reparametrizando a Loss e a Rede
9. Treinamento e Amostragem
10. Implementação e Extensões
11. Panorama

Difusão – Definição

O processo de difusão (*forward*) mapeia um exemplo x por uma série de etapas intermediárias $z_1, z_2, \dots, z_T$ via adição de ruído:

$$z_1 = \sqrt{1 - \beta_1} \cdot x + \sqrt{\beta_1} \cdot \epsilon_1$$

$$z_t = \sqrt{1 - \beta_t} \cdot z_{t-1} + \sqrt{\beta_t} \cdot \epsilon_t \quad \forall t \in \{2, 3, \dots, T\}$$

- $\epsilon_t \sim \mathcal{N}(0, I)$: ruído amostrado da normal padrão.
- Hiperparâmetros $\beta_t \in [0, 1]$ são o ***noise schedule*** (controlam a velocidade da difusão).

Difusão – Forma probabilística (Markov)

Equivalentemente, podemos escrever:

$$q(z_1 | x) = \mathcal{N}_{z_1} \left[\sqrt{1 - \beta_1} \cdot x, \beta_1 \cdot \mathbf{I} \right]$$

$$q(z_t | z_{t-1}) = \mathcal{N}_{z_t} \left[\sqrt{1 - \beta_t} \cdot z_{t-1}, \beta_t \cdot \mathbf{I} \right] \quad \forall t \geq 2$$

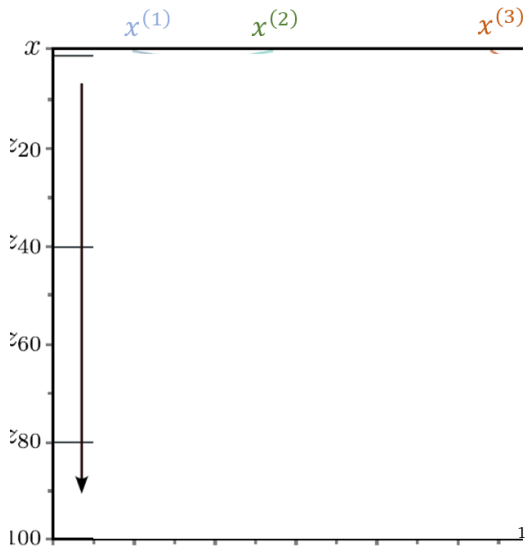
Cadeia de Markov

z_t depende apenas de z_{t-1} . Por isso, a distribuição conjunta fatoriza:

$$q(z_{1..T} | x) = q(z_1 | x) \prod_{t=2}^T q(z_t | z_{t-1}).$$

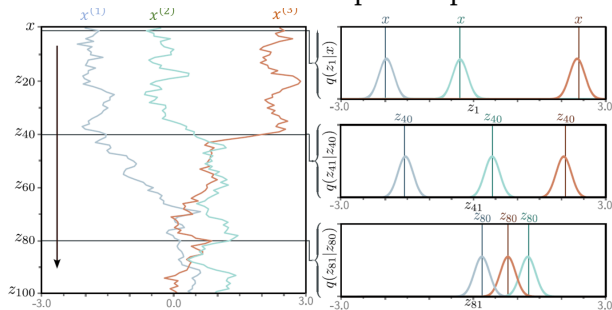
Difusão – Trajetórias 1D

- Partimos de **três instâncias** $x^{(1)}, x^{(2)}, x^{(3)}$ (1D).
- Conforme os passos de difusão são aplicados, as instâncias são combinadas com ruído.
- Após muitos passos, a distribuição converge para a distribuição do ruído $\mathcal{N}(0, I)$.



Difusão – Convergência para ruído

Repare como as instâncias vão aos poucos perdendo identidade:



15

Esquerda: trajetórias no tempo. Direita: distribuições condicionais $q(z_t)$, $q(z_t | z_{t-1})$ em três instantes.

Overview

1. Modelos Geradores – Revisão
2. Visão Geral do Processo de Difusão
3. Encoder: Processo de Difusão
- 4. Kernel de Difusão**
5. Distribuição Condicional da Difusão
6. Decoder
7. Treinamento e ELBO
8. Reparametrizando a Loss e a Rede
9. Treinamento e Amostragem
10. Implementação e Extensões
11. Panorama

Kernel de Difusão – Motivação

- Como o processo de difusão é probabilístico, para uma mesma instância x obtemos **várias amostras** z_t em cada instante.
- O processo definido até aqui é **sequencial**: para chegar em z_t precisamos passar por $z_1, \dots, z_{t-1}$.
- Para acelerar, vamos derivar uma **forma fechada** para amostrar diretamente z_t a partir de x .

Kernel de Difusão – Substituindo z_1 em z_2

Partindo das definições:

$$z_1 = \sqrt{1 - \beta_1} \cdot x + \sqrt{\beta_1} \cdot \epsilon_1, \quad z_2 = \sqrt{1 - \beta_2} \cdot z_1 + \sqrt{\beta_2} \cdot \epsilon_2.$$

Substituindo z_1 :

$$z_2 = \sqrt{1 - \beta_2} \left(\sqrt{1 - \beta_1} \cdot x + \sqrt{\beta_1} \cdot \epsilon_1 \right) + \sqrt{\beta_2} \cdot \epsilon_2.$$

Kernel de Difusão – Manipulação algébrica

Reescrevendo $\sqrt{\beta_1} = \sqrt{1 - (1 - \beta_1)}$:

$$z_2 = \sqrt{1 - \beta_2} \left(\sqrt{1 - \beta_1} x + \sqrt{1 - (1 - \beta_1)} \epsilon_1 \right) + \sqrt{\beta_2} \epsilon_2.$$

Distribuindo:

$$z_2 = \sqrt{(1 - \beta_2)(1 - \beta_1)} x + \sqrt{(1 - \beta_2) - (1 - \beta_2)(1 - \beta_1)} \epsilon_1 + \sqrt{\beta_2} \epsilon_2.$$

Kernel de Difusão – Soma de gaussianas

Lembrete

A soma de duas gaussianas independentes com médias nulas e desvios σ_a, σ_b é uma gaussiana com média zero e variância $\sigma_a^2 + \sigma_b^2$.

Os dois últimos termos viram um único ruído gaussiano:

$$z_2 = \sqrt{(1 - \beta_2)(1 - \beta_1)} \cdot x + \sqrt{1 - (1 - \beta_2)(1 - \beta_1)} \cdot \epsilon.$$

Observação

Podemos repetir esse processo para todos os z_t , sempre acumulando os fatores $(1 - \beta_s)$.

Kernel de Difusão – Forma geral

Definindo $\alpha_t = \prod_{s=1}^t (1 - \beta_s)$, temos:

$$\boxed{z_t = \sqrt{\alpha_t} \cdot x + \sqrt{1 - \alpha_t} \cdot \epsilon} \quad \epsilon \sim \mathcal{N}(0, \mathbf{I})$$

Em forma probabilística:

$$q(z_t | x) = \mathcal{N}_{z_t} \left[\sqrt{\alpha_t} \cdot x, (1 - \alpha_t) \cdot \mathbf{I} \right].$$

Vantagem prática

Podemos amostrar z_t **diretamente** a partir de x sem simular toda a cadeia. Isso permite treinamento por *minibatch* eficiente.

Exemplo Numérico: *Schedule* β e Sinal vs Ruído

Pense em x como uma imagem limpa (e.g., um pixel ou um vetor latente). β_t é a variância de ruído adicionada no passo t do *forward process*; α_t é o quanto sobrou de x após t passos. Aqui usamos $T = 4$ para visualização, DDPM real tem $T = 1000$ e β na faixa 10^{-4} a $0,02$, mas a curva é a mesma.

Schedule linear com $T = 4$: $\beta = (0,1, 0,2, 0,3, 0,4)$. Acumulamos $\alpha_t = \prod_{s=1}^t (1 - \beta_s)$:

t	α_t	$\sqrt{\alpha_t}$ (sinal)	$\sqrt{1 - \alpha_t}$ (ruído)
1	0,900	0,949	0,316
2	0,720	0,849	0,529
3	0,504	0,710	0,704
4	0,302	0,550	0,836

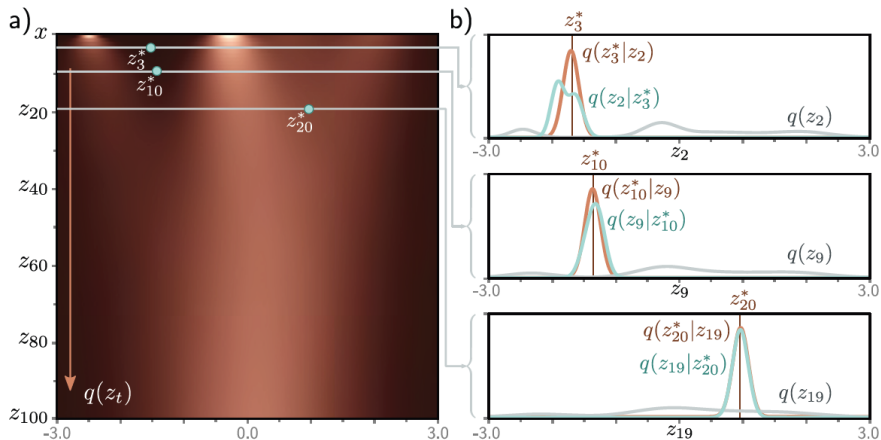
Ex.: $\alpha_3 = 0,9 \cdot 0,8 \cdot 0,7 = 0,504$.

Para $x = 1$:

$$z_t = \sqrt{\alpha_t} \cdot 1 + \sqrt{1 - \alpha_t} \cdot \epsilon$$

Em $t = 3$ o sinal e o ruído se cruzam; em $t = T$ grande, $\alpha_T \rightarrow 0$ e $z_T \approx \mathcal{N}(0, I)$ **por construção**, é o que permite o *decoder* começar a amostragem de ruído puro.

Kernel de Difusão – Visualização



Mesmo x^* pode ter diversas trajetórias; o kernel permite amostrar diretamente o resultado de t passos. Painéis à direita: densidades condicionais.

Overview

1. Modelos Geradores – Revisão
2. Visão Geral do Processo de Difusão
3. Encoder: Processo de Difusão
4. Kernel de Difusão
- 5. Distribuição Condicional da Difusão**
6. Decoder
7. Treinamento e ELBO
8. Reparametrizando a Loss e a Rede
9. Treinamento e Amostragem
10. Implementação e Extensões
11. Panorama

Distribuição condicional $q(z_{t-1} \mid z_t, x)$

Para o decoder vamos precisar de uma expressão para a difusão **revertida no tempo**, condicionada em x :

$$q(z_{t-1} \mid z_t, x) = \frac{q(z_t \mid z_{t-1}, x) q(z_{t-1} \mid x)}{q(z_t \mid x)}$$

Por regra de Bayes. Vamos manipular essa expressão.

Distribuição condicional – Simplificação Markov

- Como difusão é Markov: $q(z_t | z_{t-1}, x) = q(z_t | z_{t-1})$.
- Tirando o denominador, a igualdade vira proporcionalidade:

$$q(z_{t-1} | z_t, x) \propto q(z_t | z_{t-1}) q(z_{t-1} | x).$$

Distribuição condicional – Produto de gaussianas

Substituindo cada termo pela sua respectiva normal:

$$\propto \mathcal{N}_{z_t} \left[\sqrt{1 - \beta_t} \cdot z_{t-1}, \beta_t \mathbf{I} \right] \cdot \mathcal{N}_{z_{t-1}} \left[\sqrt{\alpha_{t-1}} \cdot x, (1 - \alpha_{t-1}) \mathbf{I} \right].$$

Lema

O produto de duas pdfs gaussianas é uma pdf gaussiana (a menos de constante).
Aplicando a fórmula de mudança de variável:

$$q(z_{t-1} \mid z_t, x) = \mathcal{N}_{z_{t-1}} \left[\frac{(1 - \alpha_{t-1})}{1 - \alpha_t} \sqrt{1 - \beta_t} z_t + \frac{\sqrt{\alpha_{t-1}} \beta_t}{1 - \alpha_t} x, \frac{\beta_t (1 - \alpha_{t-1})}{1 - \alpha_t} \mathbf{I} \right]$$

Overview

1. Modelos Geradores – Revisão
2. Visão Geral do Processo de Difusão
3. Encoder: Processo de Difusão
4. Kernel de Difusão
5. Distribuição Condicional da Difusão
- 6. Decoder**
7. Treinamento e ELBO
8. Reparametrizando a Loss e a Rede
9. Treinamento e Amostragem
10. Implementação e Extensões
11. Panorama

Decoder (*Reverse Process*)

O aprendizado está **todo no decoder**:

$$z_T \rightarrow z_{T-1} \rightarrow \cdots \rightarrow z_1 \rightarrow x.$$

- A distribuição reversa real $q(z_{t-1} | z_t)$ é complexa e potencialmente multimodal, depende dos dados.
- Na prática, **aproximamos** essa reversa por uma normal:

$$p(z_T) = \mathcal{N}_{z_T}[0, \mathbf{I}], \quad p(z_{t-1} | z_t, \boldsymbol{\theta}_t) = \mathcal{N}_{z_{t-1}}[f_t[z_t, \boldsymbol{\theta}_t], \sigma_t^2 \mathbf{I}],$$
$$p(x | z_1, \boldsymbol{\theta}_1) = \mathcal{N}_x[f_1[z_1, \boldsymbol{\theta}_1], \sigma_1^2 \mathbf{I}].$$

Quando essa aproximação é razoável

Se o número de passos T for grande e β_t for próximo de 0, cada passo da reversa é quase determinístico e a aproximação gaussiana é boa.

Overview

1. Modelos Geradores – Revisão
2. Visão Geral do Processo de Difusão
3. Encoder: Processo de Difusão
4. Kernel de Difusão
5. Distribuição Condicional da Difusão
6. Decoder
- 7. Treinamento e ELBO**
8. Reparametrizando a Loss e a Rede
9. Treinamento e Amostragem
10. Implementação e Extensões
11. Panorama

Treinamento – O problema

Maximizar a log-verossimilhança dos dados é o objetivo:

$$\hat{\theta}_{1...T} = \operatorname{argmax}_{\theta_{1...T}} \left[\sum_{i=1}^N \log p(x^{(i)} \mid \theta_{1...T}) \right]$$

$$p(x \mid \theta_{1...T}) = \int p(x, z_{1...T} \mid \theta_{1...T}) dz_{1...T}$$

Inviável

Essa integral é **intratável**. Vamos usar o **ELBO** como substituto.

Treinamento – Distribuições conjuntas

Conjunta do modelo (decoder)

$$p(x, z_{1...T} \mid \theta_{1...T}) = p(x \mid z_1, \theta_1) \prod_{t=2}^T p(z_{t-1} \mid z_t, \theta_t) \cdot p(z_T)$$

Conjunta da difusão (encoder)

$$q(z_{1...T} \mid x) = q(z_1 \mid x) \prod_{t=2}^T q(z_t \mid z_{t-1})$$

Tudo gaussiano $\Rightarrow$ tudo tratável passo a passo.

Evidence Lower Bound (ELBO)

Multiplicamos e dividimos por $q(z_{1...T} | x)$, e aplicamos a desigualdade de Jensen:

$$\begin{aligned}\log p(x | \boldsymbol{\theta}_{1...T}) &= \log \left[\int p(x, z_{1...T} | \boldsymbol{\theta}_{1...T}) dz \right] \\ &= \log \left[\int q(z_{1...T} | x) \frac{p(x, z_{1...T} | \boldsymbol{\theta}_{1...T})}{q(z_{1...T} | x)} dz \right] \\ &\geq \int q(z_{1...T} | x) \log \left[\frac{p(x, z_{1...T} | \boldsymbol{\theta}_{1...T})}{q(z_{1...T} | x)} \right] dz \quad (\text{Jensen})\end{aligned}$$

Vamos trabalhar nesse termo (o ELBO).

ELBO – Substituindo as conjuntas

Substituindo as definições do encoder e decoder:

$$\begin{aligned} & \log \left[\frac{p(x|z_1, \boldsymbol{\theta}_1) \prod_{t=2}^T p(z_{t-1}|z_t, \boldsymbol{\theta}_t) \cdot p(z_T)}{q(z_1|x) \prod_{t=2}^T q(z_t|z_{t-1})} \right] \\ &= \log \left[\frac{p(x|z_1, \boldsymbol{\theta}_1)}{q(z_1|x)} \right] + \log \left[\frac{\prod_{t=2}^T p(z_{t-1}|z_t, \boldsymbol{\theta}_t)}{\prod_{t=2}^T q(z_t|z_{t-1})} \right] + \log p(z_T). \end{aligned}$$

ELBO – Aplicando Bayes em $q(z_t|z_{t-1})$

Pela regra de Bayes (e Markov):

$$q(z_t | z_{t-1}) = \frac{q(z_{t-1} | z_t, x) q(z_t | x)}{q(z_{t-1} | x)}.$$

Aplicando isso na razão dentro do produtório, telescópios fazem com que sobrevivam apenas $q(z_1 | x)$ no numerador e $q(z_T | x)$ no denominador:

$$\prod_{t=2}^T \frac{1}{q(z_t | z_{t-1})} = \prod_{t=2}^T \frac{q(z_{t-1} | x)}{q(z_{t-1} | z_t, x) q(z_t | x)}.$$

ELBO – Simplificação telescópica

Após o cancelamento telescópico, a expressão fica:

$$\begin{aligned} &= \log \left[\frac{p(x|z_1, \boldsymbol{\theta}_1) q(z_1|x)}{q(z_1|x)} \right] + \log \left[\frac{\prod_{t=2}^T p(z_{t-1}|z_t, \boldsymbol{\theta}_t)}{\prod_{t=2}^T q(z_{t-1}|z_t, x)} \right] + \log \left[\frac{p(z_T)}{q(z_T|x)} \right] \\ &= \log p(x|z_1, \boldsymbol{\theta}_1) + \log \left[\frac{\prod_{t=2}^T p(z_{t-1}|z_t, \boldsymbol{\theta}_t)}{\prod_{t=2}^T q(z_{t-1}|z_t, x)} \right] + \cancel{\log \left[\frac{p(z_T)}{q(z_T|x)} \right]} \approx 0 \end{aligned}$$

O último termo é desprezível: $q(z_T|x) \approx p(z_T)$ para T grande.

ELBO – Formulação final

Tomando esperança em $q(z_{1...T} | x)$:

$$\text{ELBO}[\boldsymbol{\theta}_{1...T}] = \mathbb{E}_{q(z_1|x)}[\log p(x|z_1, \boldsymbol{\theta}_1)] - \sum_{t=2}^T \mathbb{E}_{q(z_t|x)}[D_{KL}[q(z_{t-1}|z_t, x) \parallel p(z_{t-1}|z_t, \boldsymbol{\theta}_t)]]$$

- **Termo de reconstrução**: $\log p(x|z_1, \boldsymbol{\theta}_1)$ é uma normal $\Rightarrow$ MSE.
- **Termos KL**: ambas distribuições são normais $\Rightarrow$ **forma fechada!**

ELBO – KL entre normais

A KL entre duas gaussianas tem expressão fechada. Aplicando-a:

KL desenvolvida

$$\mathbb{E}_{q(z_t|x)} [D_{KL}(q \parallel p)] = \frac{1}{2\sigma_t^2} \left| \frac{1-\alpha_{t-1}}{1-\alpha_t} \sqrt{1-\beta_t} z_t + \frac{\sqrt{\alpha_{t-1}} \beta_t}{1-\alpha_t} x - f_t[z_t, \theta_t] \right|^2$$

Substituindo na expressão do ELBO obtemos a **Diffusion Loss Function**.

Diffusion Loss Function

$$\mathcal{L}[\boldsymbol{\theta}_{1...T}] = \underbrace{\sum_{i=1}^I (-\log \mathcal{N}_x[f_1[z_1, \boldsymbol{\theta}_1], \sigma_1^2 \mathbf{I}])}_{\text{reconstrução}} + \underbrace{\sum_{t=2}^T \frac{1}{2\sigma_t^2} \left| \frac{1-\alpha_{t-1}}{1-\alpha_t} \sqrt{1-\beta_t} z_t + \frac{\sqrt{\alpha_{t-1}} \beta_t}{1-\alpha_t} x - f_t[z_t, \boldsymbol{\theta}_t] \right|^2}_{\text{erro quadrático no timestep } t}$$

- Forma de **MSE** em vez de log-verossimilhança intratável.
- Cada termo t pede a rede f_t a prever a média da reversa em z_{t-1} .

Overview

1. Modelos Geradores – Revisão
2. Visão Geral do Processo de Difusão
3. Encoder: Processo de Difusão
4. Kernel de Difusão
5. Distribuição Condicional da Difusão
6. Decoder
7. Treinamento e ELBO
- 8. Reparametrizando a Loss e a Rede**
9. Treinamento e Amostragem
10. Implementação e Extensões
11. Panorama

Porém... um detalhe empírico

Observação experimental

Após treinar diversos modelos difusores que tentavam descobrir diretamente a imagem, percebeu-se empiricamente que os resultados eram **melhores** ao usar a rede para **estimar o ruído** ϵ adicionado, e não a imagem.

Vamos reparametrizar a função de custo para que a rede prediga ϵ em vez de z_{t-1} .

Reparametrizando a loss – Isolar x

Do kernel de difusão $z_t = \sqrt{\alpha_t} x + \sqrt{1 - \alpha_t} \epsilon$:

$$x = \frac{z_t}{\sqrt{\alpha_t}} - \frac{\sqrt{1 - \alpha_t}}{\sqrt{\alpha_t}} \epsilon.$$

Substituímos essa expressão de x dentro da loss e simplificamos.

Reparametrizando a loss – Simplificação

Após álgebra, o argumento dentro do quadrado vira:

$$\frac{1}{\sqrt{1-\beta_t}} z_t - \frac{\beta_t}{\sqrt{1-\alpha_t}\sqrt{1-\beta_t}} \epsilon = f_t[z_t, \theta_t].$$

Reparametrização da rede

Definimos $f_t[z_t, \theta_t]$ na mesma forma:

$$f_t[z_t, \theta_t] = \frac{1}{\sqrt{1-\beta_t}} z_t - \frac{\beta_t}{\sqrt{1-\alpha_t}\sqrt{1-\beta_t}} g_t[z_t, \theta_t],$$

onde g_t é a **nova rede que prediz o ruído**.

Reparametrizando a rede – Expansão intermediária

Substituindo f_t (reparametrizada) e $x = \frac{z_t}{\sqrt{\alpha_t}} - \frac{\sqrt{1-\alpha_t}}{\sqrt{\alpha_t}} \epsilon$ no termo da loss:

$$\left| \frac{1-\alpha_{t-1}}{1-\alpha_t} \sqrt{1-\beta_t} z_t + \frac{\sqrt{\alpha_{t-1}} \beta_t}{1-\alpha_t} \left(\frac{z_t}{\sqrt{\alpha_t}} - \frac{\sqrt{1-\alpha_t}}{\sqrt{\alpha_t}} \epsilon_t \right) - \frac{1}{\sqrt{1-\beta_t}} z_t + \frac{\beta_t}{\sqrt{1-\alpha_t} \sqrt{1-\beta_t}} g_t[z_t, \theta_t] \right|^2$$

Consolidando termos em z_t

Coletando os coeficientes de z_t e usando $\alpha_t = (1 - \beta_t)\alpha_{t-1}$, os termos em z_t e x **se cancelam**. Sobra apenas algo proporcional a $(g_t - \epsilon_t)$.

Reparametrizando a rede – Loss final

Substituindo, o termo do quadrado vira simplesmente $|g_t[z_t, \theta_t] - \epsilon_t|^2$ a menos de uma constante multiplicativa, que podemos absorver durante a otimização:

$$\mathcal{L}[\theta_{1...T}] = \sum_{i=1}^I \sum_{t=2}^T |g_t[z_t, \theta_t] - \epsilon_t|^2$$

Função de custo do modelo difusor

É um erro quadrático médio entre o ruído real ϵ_t adicionado em cada passo e a estimativa g_t da rede. **Surpreendentemente simples.**

O termo de reconstrução ($t = 1$) tem a mesma forma de MSE em ϵ ; na prática o somatório roda de $t = 1$ a T .

Reparametrizando a rede – O que a rede prevê?

A rede pode ser treinada para prever alvos **equivalentes**; a escolha afeta a estabilidade e a qualidade:

- **ϵ -prediction** (DDPM, usado aqui): estima o ruído ϵ . Loss MSE simples e estável; é o default.
- **x_0 -prediction**: estima diretamente a imagem limpa x . Equivalente pela álgebra do kernel, mas empiricamente pior em timesteps de ruído alto.
- **v -prediction** (Salimans & Ho, 2022): estima $v = \sqrt{\alpha_t} \epsilon - \sqrt{1 - \alpha_t} x$, combinação de ruído e sinal. Estável em todo o schedule; comum em geração de vídeo e em distillation.

Resumo

As três são equivalentes via o kernel $z_t = \sqrt{\alpha_t} x + \sqrt{1 - \alpha_t} \epsilon$; diferem na escala efetiva da loss por timestep.

Overview

1. Modelos Geradores – Revisão
2. Visão Geral do Processo de Difusão
3. Encoder: Processo de Difusão
4. Kernel de Difusão
5. Distribuição Condicional da Difusão
6. Decoder
7. Treinamento e ELBO
8. Reparametrizando a Loss e a Rede
- 9. Treinamento e Amostragem**
10. Implementação e Extensões
11. Panorama

Algoritmo 18.1: Treinamento

Algorithm 18.1: Diffusion model training

Input: Training data $\mathbf{x}$

Output: Model parameters ϕ_t

repeat

 for $i \in \mathcal{B}$ do // For every training example index in batch
 $t \sim \text{Uniform}[1, \dots, T]$ // Sample random timestep
 $\epsilon \sim \text{Norm}[\mathbf{0}, \mathbf{I}]$ // Sample noise
 $\ell_i = \left\| g_t \left[\sqrt{\alpha_t} \mathbf{x}_i + \sqrt{1 - \alpha_t} \epsilon, \phi_t \right] - \epsilon \right\|^2$ // Compute individual loss

 Accumulate losses for batch and take gradient step

until converged

- Para cada x_i no *minibatch*, sorteie t uniforme.
- Sorteie $\epsilon \sim \mathcal{N}(0, \mathbf{I})$, monte z_t pelo kernel.
- Calcule $\ell_i = \|g_t[z_t, \theta_t] - \epsilon\|^2$.
- Acumule e dê um passo de gradiente.

Algoritmo 18.2: Amostragem

Algorithm 18.2: Sampling

Input: Model, $\mathbf{g}_t[\bullet, \phi_t]$

Output: Sample, $\mathbf{x}$

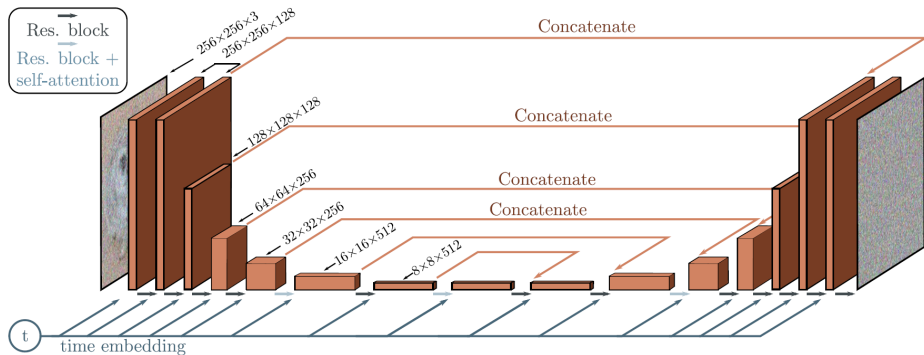
```
 $\mathbf{z}_T \sim \text{Norm}_{\mathbf{z}}[\mathbf{0}, \mathbf{I}]$  // Sample last latent variable
for  $t = T \dots 2$  do
     $\hat{\mathbf{z}}_{t-1} = \frac{1}{\sqrt{1-\beta_t}} \mathbf{z}_t - \frac{\beta_t}{\sqrt{1-\alpha_t}\sqrt{1-\beta_t}} \mathbf{g}_t[\mathbf{z}_t, \phi_t]$  // Predict previous latent variable
     $\epsilon \sim \text{Norm}_{\epsilon}[\mathbf{0}, \mathbf{I}]$  // Draw new noise vector
     $\mathbf{z}_{t-1} = \hat{\mathbf{z}}_{t-1} + \sigma_t \epsilon$  // Add noise to previous latent variable
 $\mathbf{x} = \frac{1}{\sqrt{1-\beta_1}} \mathbf{z}_1 - \frac{\beta_1}{\sqrt{1-\alpha_1}\sqrt{1-\beta_1}} \mathbf{g}_1[\mathbf{z}_1, \phi_1]$  // Generate sample from  $\mathbf{z}_1$  without noise
```

- Comece com $\mathbf{z}_T \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$.
- Para $t = T, \dots, 2$: prediga $\mathbf{z}_{t-1}$ usando $\mathbf{g}_t$ e adicione um pouco de ruído.
- Para o último passo, gere $\mathbf{x}$ sem ruído.

Overview

1. Modelos Geradores – Revisão
2. Visão Geral do Processo de Difusão
3. Encoder: Processo de Difusão
4. Kernel de Difusão
5. Distribuição Condicional da Difusão
6. Decoder
7. Treinamento e ELBO
8. Reparametrizando a Loss e a Rede
9. Treinamento e Amostragem
- 10. Implementação e Extensões**
11. Panorama

Modelo para estimar ϵ_t – *U-Net*



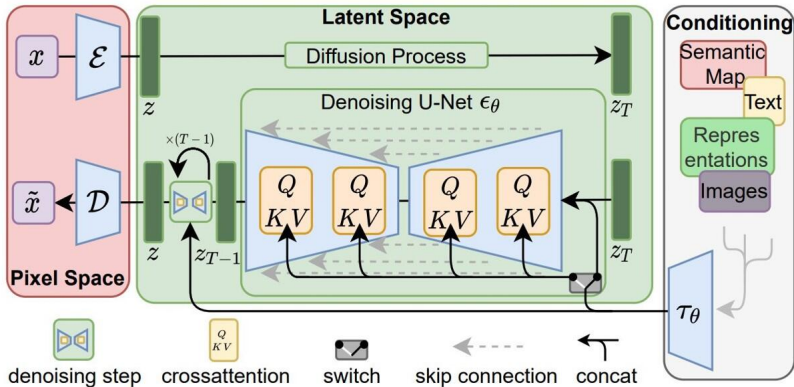
- Arquitetura padrão: **U-Net** com blocos residuais e auto-atenção.
- Recebe z_t + um time embedding de t , devolve uma estimativa de ϵ com a mesma forma de z_t .
- *Skip connections* preservam alta-frequência entre escalas.

Classifier-Free Diffusion Guidance²

- É comum adicionar uma **informação condicionante** no processo de geração.
- Pode parecer contraintuitivo, contudo informações sobre a classe que queremos gerar **auxiliam o processo** e geram imagens com maior qualidade.
- O treinamento condicionado é feito adicionando algum tipo de **embedding** da classe à U-Net.
 - Por exemplo, embeddings do BERT sobre a descrição da imagem (texto-para-imagem).
- Classifier-Free: durante o treino, esporadicamente “descondicionamos” a rede; na geração, interpolamos entre versões condicionada e incondicional para controlar a força da condição.

²Jonathan Ho e Tim Salimans. “Classifier-Free Diffusion Guidance”. Em: [arXiv preprint arXiv:2207.12598](https://arxiv.org/abs/2207.12598) (2022).

Latent Diffusion Models³



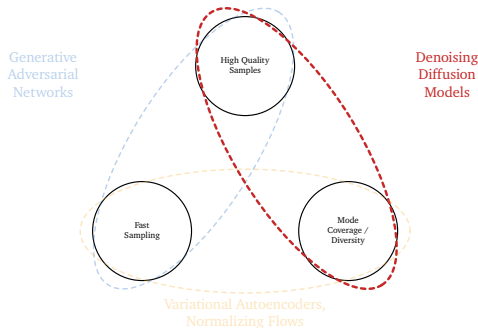
- Aplique a difusão no **espaço latente** de um VAE, não em pixels.
- Reduz drasticamente o custo computacional (latente é $\sim 64 \times 64$ em vez de 1024×1024).
- Base de *Stable Diffusion*.

Overview

1. Modelos Geradores – Revisão
2. Visão Geral do Processo de Difusão
3. Encoder: Processo de Difusão
4. Kernel de Difusão
5. Distribuição Condicional da Difusão
6. Decoder
7. Treinamento e ELBO
8. Reparametrizando a Loss e a Rede
9. Treinamento e Amostragem
10. Implementação e Extensões
- 11. Panorama**

Onde modelos difusores ficam no trilema?

- **Qualidade visual:** ✓✓ (estado-da-arte em imagens).
- **Cobertura / diversidade:** ✓ (treino por NLL / ELBO cobre a distribuição toda).
- **Amostragem rápida:** × (tipicamente $T = 1000$ passos pela U-Net; técnicas como DDIM e consistency models atacam esse problema).



Posicionamento

Modelos difusores ocupam o canto qualidade + cobertura do trilema. Para aplicações que exigem latência baixa, *Latent Diffusion* + amostradores rápidos são o compromisso padrão.

Resumo

- **Difusão** (encoder, sem parâmetros): cadeia de Markov gaussiana que destrói gradualmente a estrutura de x .
- **Kernel**: forma fechada para amostrar z_t diretamente de x .
- **Reverse process** (decoder): rede neural aprende a reverter um passo da difusão.
- **ELBO**: termo de reconstrução + soma de KLs entre normais (todas com forma fechada).
- **Reparametrização**: prever ruído ϵ em vez da imagem leva a uma loss MSE simples.
- **Arquitetura**: U-Net com *time embedding*.
- **Extensões**: *classifier-free guidance*, *latent diffusion* (Stable Diffusion).

Leituras Recomendadas

- Capítulo 18: Simon J. D. Prince. Understanding Deep Learning. MIT Press, 2023
- Jonathan Ho, Ajay Jain e Pieter Abbeel. “Denoising Diffusion Probabilistic Models”. Em: NeurIPS. vol. 33. 2020, pp. 6840–6851
- Jonathan Ho e Tim Salimans. “Classifier-Free Diffusion Guidance”. Em: arXiv preprint arXiv:2207.12598 (2022)
- Robin Rombach et al. “High-Resolution Image Synthesis with Latent Diffusion Models”. Em: CVPR. 2022, pp. 10684–10695
- Jascha Sohl-Dickstein et al. “Deep Unsupervised Learning using Nonequilibrium Thermodynamics”. Em: ICML. 2015, pp. 2256–2265

Referências I

- [1] Jonathan Ho, Ajay Jain e Pieter Abbeel. “Denoising Diffusion Probabilistic Models”. Em: [NeurIPS](#). Vol. 33. 2020, pp. 6840–6851.
- [2] Jonathan Ho e Tim Salimans. “Classifier-Free Diffusion Guidance”. Em: [arXiv preprint arXiv:2207.12598](#) (2022).
- [3] Simon J. D. Prince. [Understanding Deep Learning](#). MIT Press, 2023.
- [4] Robin Rombach et al. “High-Resolution Image Synthesis with Latent Diffusion Models”. Em: [CVPR](#). 2022, pp. 10684–10695.
- [5] Jascha Sohl-Dickstein et al. “Deep Unsupervised Learning using Nonequilibrium Thermodynamics”. Em: [ICML](#). 2015, pp. 2256–2265.